“Carpool Ride Sharing App.”

‘CSCI313 Project’

By:

- Abdulhady Ibrahim Abdulsattar _ 211000768
- Ahmed Sameh Awad _ 211000422
- Youssef Salem Hassan – 211000582
- Mohamed omar helmy _ 211001774
- Mohamed Hossam Hassan _ 211000405
- Ahmed Mohamed Ahmed _ 211001311

UNDER THE SUPERVISION OF

Dr.Ahmed Fathy El Nokrashy

Table of Contents

1. Introduction

1.1 Purpose

This document offers a full overview of the Carpool Ride Sharing App project, specifically designed for university students. It details the scope and technologies used in this project while defining its target audience. Additionally, it also provides an inclusive summary of all information contained within the entire text to allow easier understanding.

1.2 Project Scope

The Carpool Ride Sharing App is a free, convenient tool created specifically for university students. It provides an organized platform to facilitate and coordinate ride shares within the campus community.

The app's primary objectives include:

- Allowing students to find and share rides for daily commutes, trips to campus, or other common destinations.

- Enhancing transportation efficiency and reducing fuel consumption.

1.3 Used Technologies

- Flutter
- Dart
- Firebase
- Google Maps API
- Figma

1.4 Intended Audience

The Carpool Ride Sharing App is mainly designed for college students. Students looking for practical commute options are among the target audience's demographics, both undergraduate and graduate.

- Faculty and staff from the university who are open to carpooling or offering rides to students.

- University officials who might encourage or endorsing the app's use on campus.

1.5 Overview of the document

This document is divided into a number of sections, each of which focuses on a different aspect of the carpool ride-sharing app created for college students. The app's scope, technological stack, target market, main features, and user guidelines will all be covered in more detail in the sections that follow.

2. Overall Description (Chapter 2):

In this chapter, we present a comprehensive overview of the Carpooling Ride Sharing App project, highlighting its product perspective, user characteristics, constraints, and assumptions and dependencies.

2.1 Product Perspective

The Carpool Ride Sharing App operates as a stand-alone system designed to connect university students for ride-sharing purposes. It does not rely on or integrate with other external systems or applications. The primary functions of the app are as follows:

2.2 Product Function

- Facilitating the matching of students seeking rides with those offering rides for daily commutes, trips to campus, and common destinations.

- Providing a user-friendly interface for creating and managing ride-sharing requests and offers.

- Implementing a rating and review system to enhance user trust.

- Utilizing GPS and the Google Maps API to assist with route navigation.

- Sharing with users details for pickup and drop-off.

2.3 User Characteristics

The target users of the Carpool Ride Sharing App mainly consist of university students, including both undergraduate and graduate students. Additionally, the app is open to the following groups:

- Faculty and staff members of the university who are willing to carpool with students or offer rides.

- University officials who may promote or endorse the app for campus-wide use.

2.4 Constraints

There are certain constraints that need to be considered during the development of the Carpool Ride Sharing App:

- The app is designed exclusively for university students, limiting its user base.

- Data privacy and security must be ensured to protect user information.

- Adherence to local transportation regulations and laws is crucial.

- The app must be compatible with Android and iOS platforms.

- Limited resources are available for development and maintenance.

2.5 Assumptions and Dependencies

To ensure the success of this project, we make certain assumptions and identify key dependencies:

- Assumption: Users will have smartphones running Android or iOS.

- Assumption: Users will have access to a stable internet connection.

- Dependency: The app relies on Firebase for user authentication and real-time database services.

- Dependency: Google Maps API is essential for mapping and navigation functionalities.

- Dependency: Figma is used for designing and prototyping the user interface.

3. Functional Requirements (Chapter 3)

3.1 User Class 1: The Passenger

3.1.1 Functional Requirements

Title: User Registration

Description: The system will allow the user to create an account to book and manage rides.

Required Information:

- Full Name
- Email (unregistered email)
- Phone Number
- Payment Information (Credit Card or other preferred payment methods)

3.1.2 Functional Requirements

Title: Login

Description: The system will allow the user to log in using their registered email and password.

3.1.3 Functional Requirements

Title: Book a Ride

Description: The system will enable the user to request a ride by providing the following details:

Pickup Location

Drop-off Location

Preferred Car Type (if available)

Special Requests (e.g., carpooling preferences)

3.1.4 Functional Requirements

Title: Cancel Ride

Description: The system will allow the user to cancel a booked ride within a specified time window.

3.1.5 Functional Requirements

Title: Track Ride

Description: The system will provide real-time tracking information for the user to monitor the location and estimated time of arrival of the assigned driver.

3.1.6 Functional Requirements

Title: Rate and Review Driver

Description: After the completion of the ride, the system will prompt the user to rate and leave a review for the driver.

3.1.7 Functional Requirements

Title: View Ride History

Description: The system will allow the user to view their ride history, including details such as date, time, locations, and fare.

3.1.8 Functional Requirements

Title: Notifications

Description: The system will notify the user of ride confirmation, driver details, and any important updates related to their booked rides.

3.1.9 Functional Requirements

Title: Emergency Assistance

Description: The system will include an emergency button or feature that allows users to quickly contact emergency services or share their live location with trusted contacts in case of emergency during a ride.

3.2 User Class 2: The Driver

3.2.1 Functional Requirements

Title: Driver Registration

Description: The system will allow individuals to sign up as drivers by providing the necessary information.

Required Information:

- Full Name
- Email (unregistered email)
- Phone Number
- Driver's License Information
- Vehicle Details (Make, Model, Plate Number)
- Profile Photo

3.2.2 Functional Requirements

Title: Accept/Decline Ride Requests

Description: The system will notify the driver of ride requests and allow them to accept or decline based on their availability.

3.2.3 Functional Requirements

Title: View Earnings

Description: The system will allow the driver to view their earnings, including details of completed rides, tips, and any additional charges.

3.2.4 Functional Requirements

Title: Navigation

Description: The system will integrate navigation features to assist drivers in reaching the pickup and drop-off locations efficiently.

3.2.5 Functional Requirements

Title: Update Availability

Description: The system will allow the driver to update their availability status (online or offline) to receive or pause ride requests.

3.2.6 Functional Requirements

Title: Rate and Review Passenger

Description: After completing a ride, the system will prompt the driver to rate and leave a review for the passenger.

3.2.7 Functional Requirements

Title: Notifications

Description: The system will notify the driver of new ride requests, important updates, and other relevant information.

3.2.8 Functional Requirements

Title: Emergency Assistance

Description: The system will include an emergency button or feature that allows drivers to quickly contact emergency services or report safety concerns.

4. Non-Functional Requirements (Chapter 4)

4.1 Reliability

- The system is expected to have a mean time between failures (MTBF) of at least 500 hours.
- In the event of a system failure, the time required for recovery should not exceed 4 hours.

4.2 Recoverability

- The system should have a backup and recovery mechanism in place to restore data and functionality within 4 hours of a breakdown.
- Backup data should be stored securely and be easily accessible for recovery purposes.

4.3 Performance

- The app's response time for key actions (e.g., booking a ride, tracking a ride) should not exceed 3 seconds.
- The system should be able to handle concurrent user requests efficiently, supporting a minimum of 1000 simultaneous users.

4.4 Availability

- The system should be designed for continuous availability, capable of running 24/7, provided there is a stable internet connection.
- Scheduled maintenance activities, if necessary, should be communicated in advance, and downtime should be minimized.

4.5 Usability

- Users should be able to learn and navigate the app's basic functions within 30 minutes of use.
- Both passengers and drivers should be able to complete a ride (from booking to completion) within 1 hour for the first-time use, with subsequent rides taking less time due to familiarity.

4.6 Maintainability

- The system architecture should allow for seamless integration of future enhancements and additional features.
- Codebase and database should be well-documented to facilitate easy maintenance and updates.
- Regular system updates and maintenance should be performed without causing disruption to the user experience.

5. Interface

5.1 System Interface

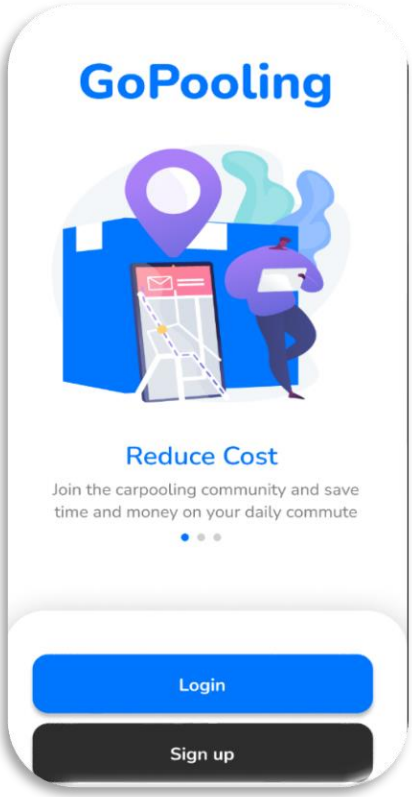


Figure 1

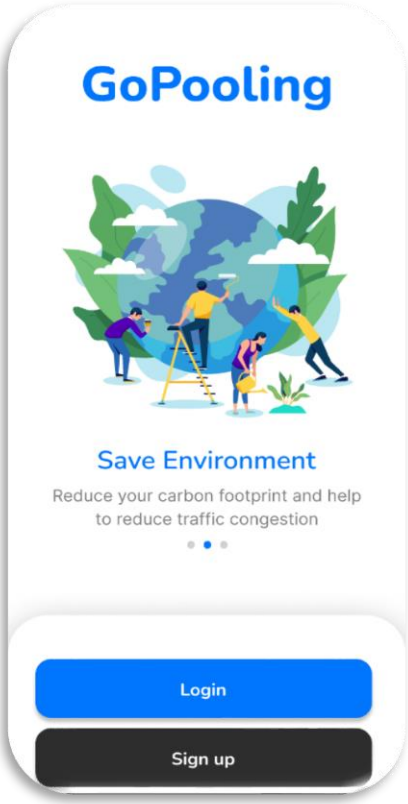


Figure 2

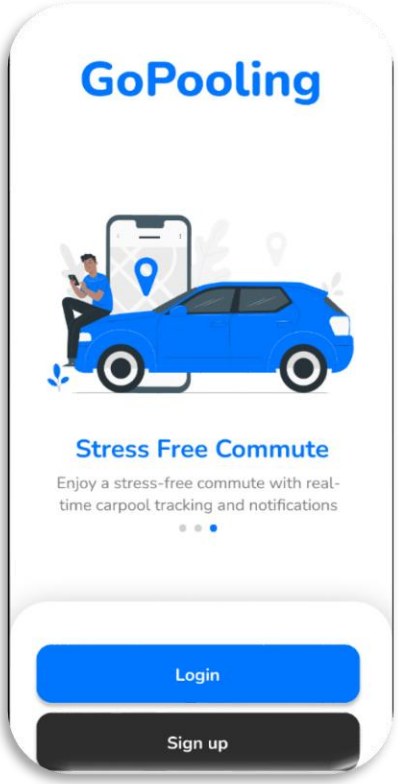


Figure 3

Figure 1, Figure 2 and Figure 3 serve as the initial welcome screen for users accessing the carpool application. It provides a visually appealing introduction to the platform, setting a positive tone for the user experience. highlights the login interface, presenting users with the opportunity to access their accounts and utilize the carpool features. displays the sign-up interface, encouraging unpracticed users to register for the carpool application.

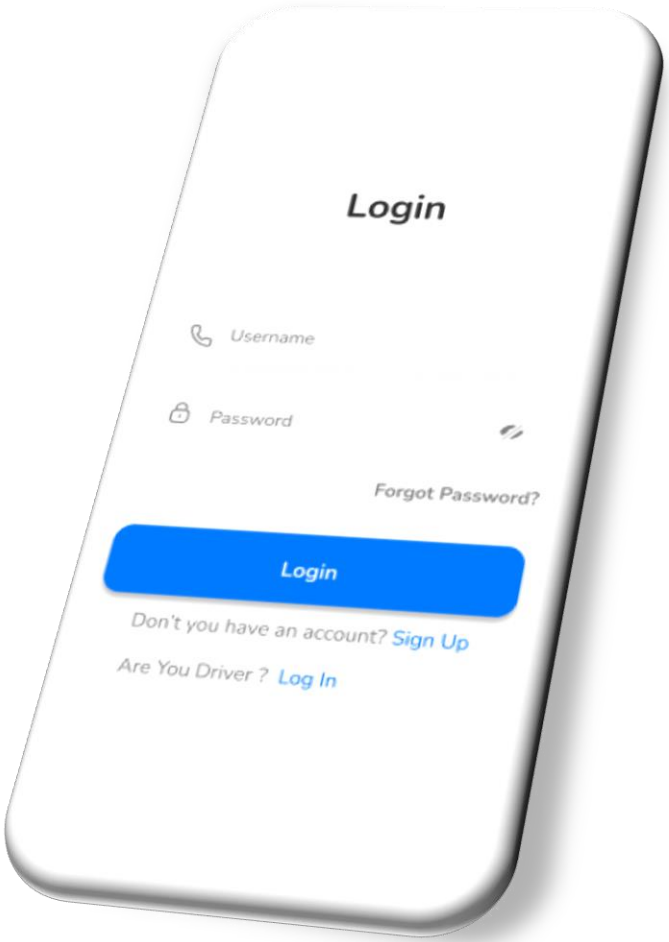


Figure 4

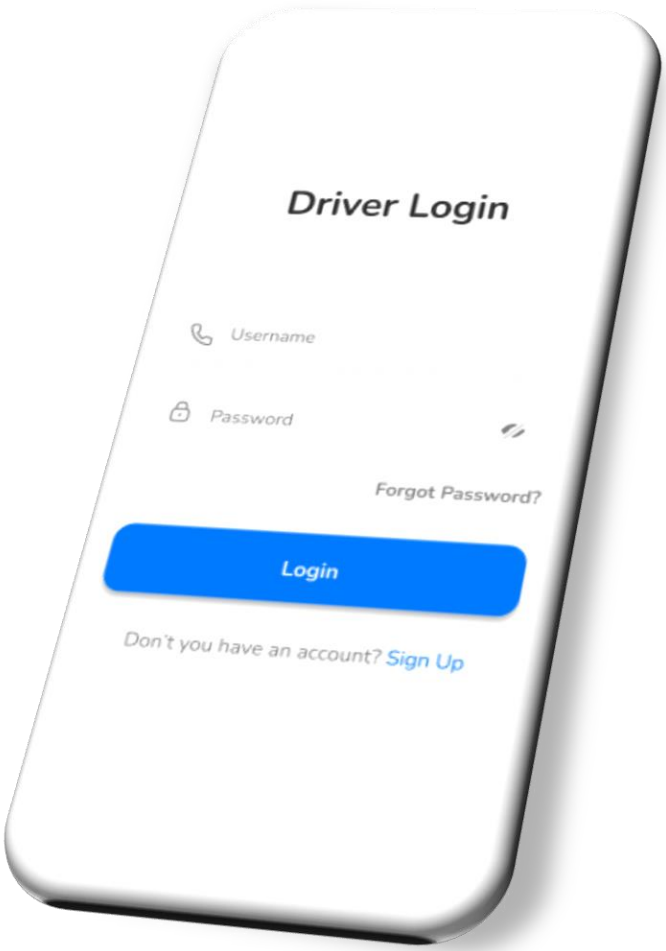


Figure 5

Upon launching the app for the first time, one shall come across a login page. Users will be presented with two Log in options: either you can log in as a “User” or a “Driver.”

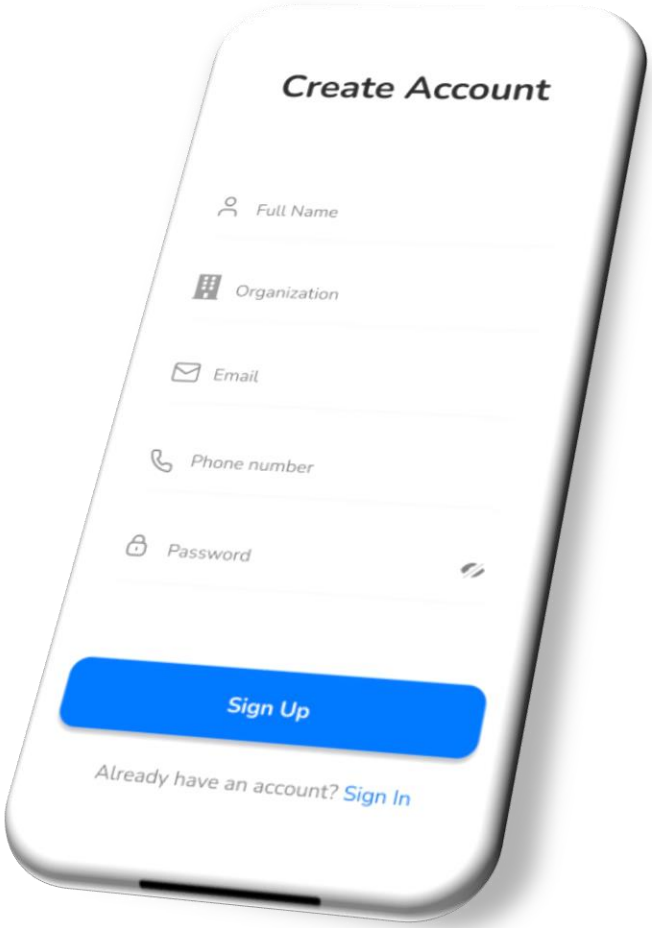


Figure 6

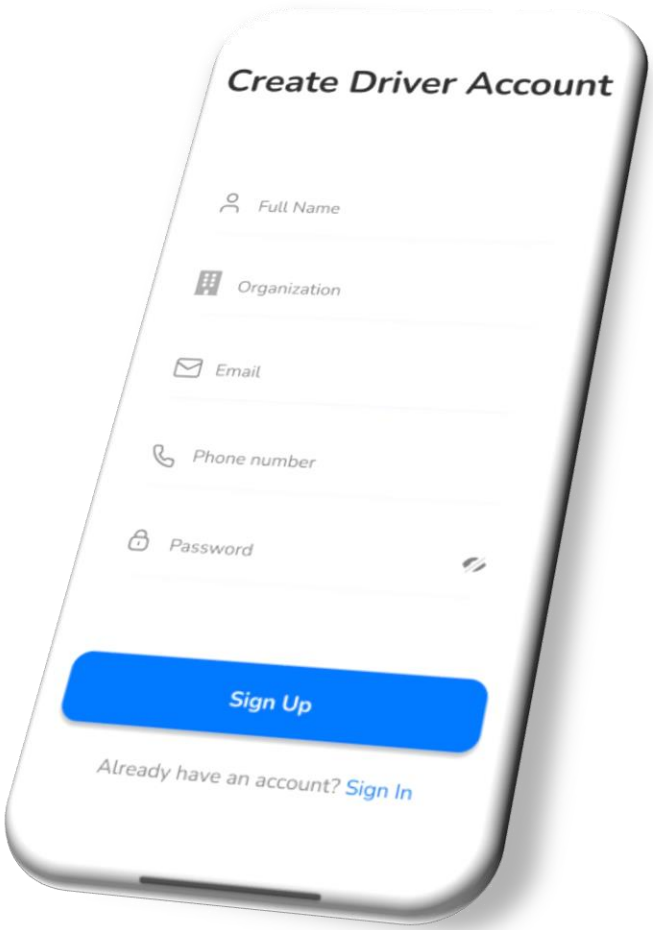


Figure 7

After selecting the option of “Sign up” user will have to enter their names, passwords, e-mail addresses, and organization to create an account, likewise to a driver.

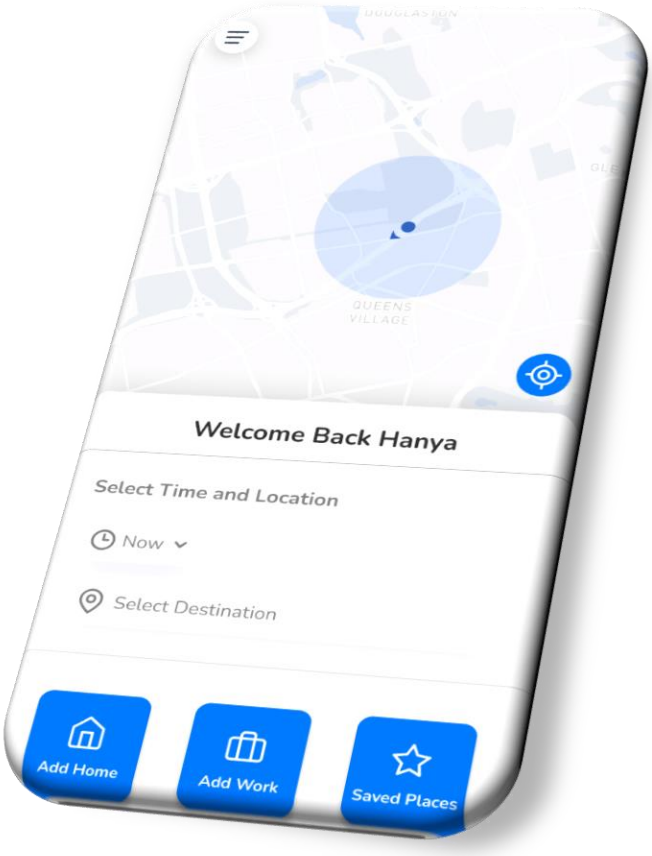


Figure 8

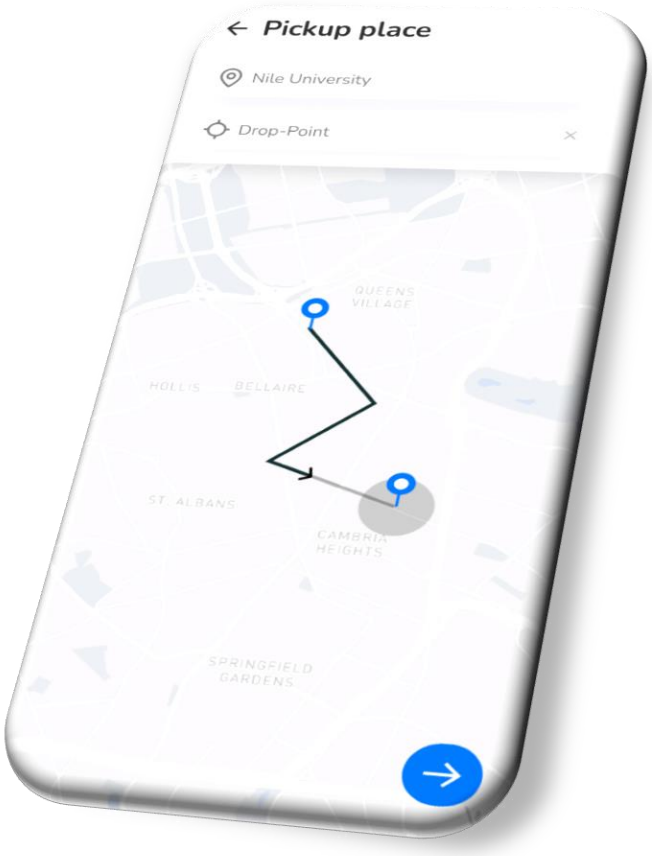
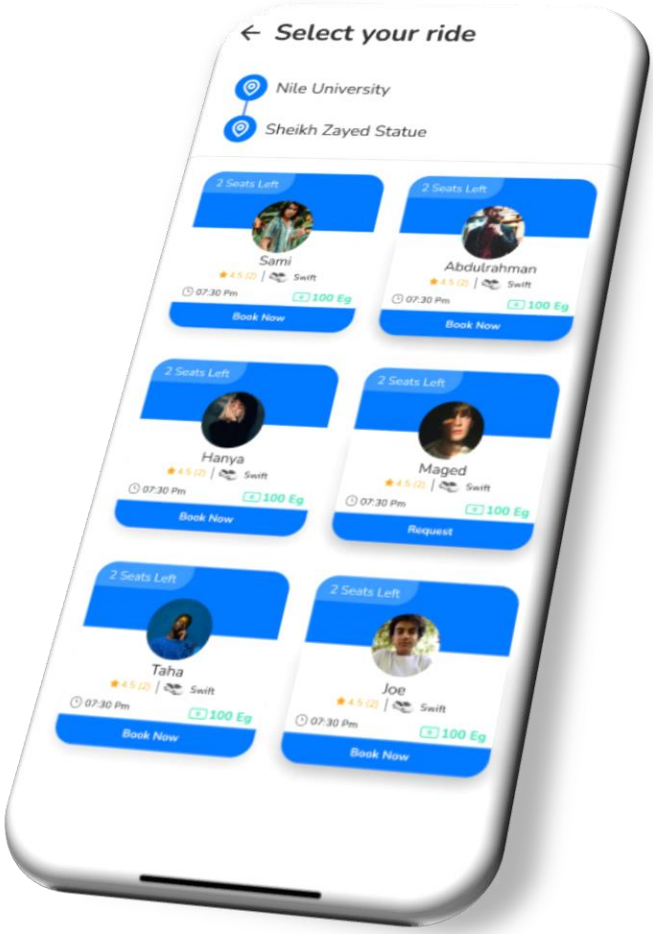


Figure 9

Figure 8 Provides an overview of the ridesharing application`s home page, showing key features such as maps, pick-up and drop-off locations, adding a home, adding a workspace, and dedicated sections for managing saved locations.

Figure 9 The interface zooms in on the pickup and destination selector, allowing users to easily select their points on the map. A clear confirm button is provided after selecting the pickup and destination, ensuring accuracy and ease of use.



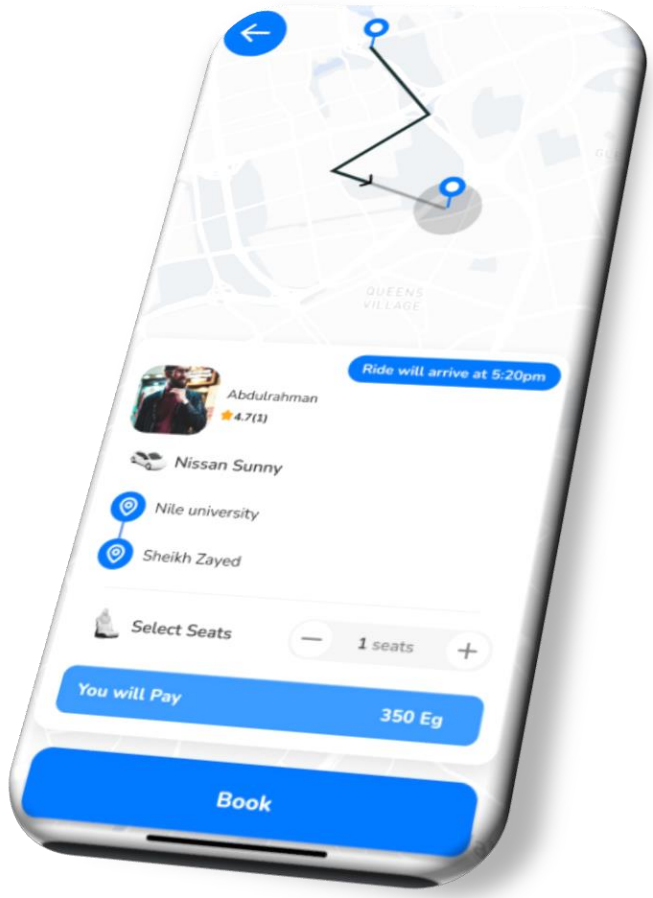


Figure 10

Figure 11

Figure 10: Upon deciding your destination, you may choose which date and pick up time you want, and also you can also choose which driver or car you want to book.

Figure 11: After Choosing your ride a final confirmation page-like will appear to you with details like your destination, fair, and captain details.

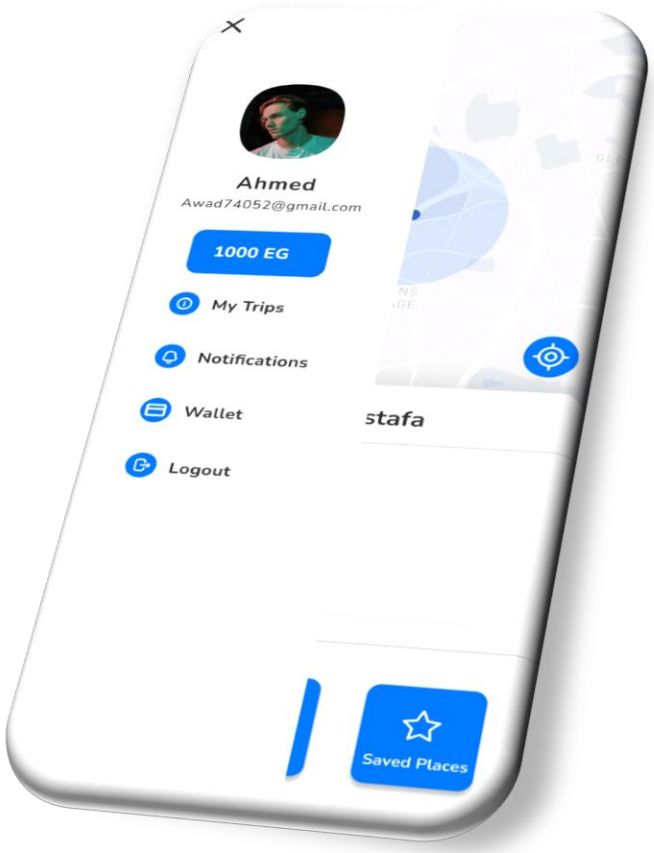
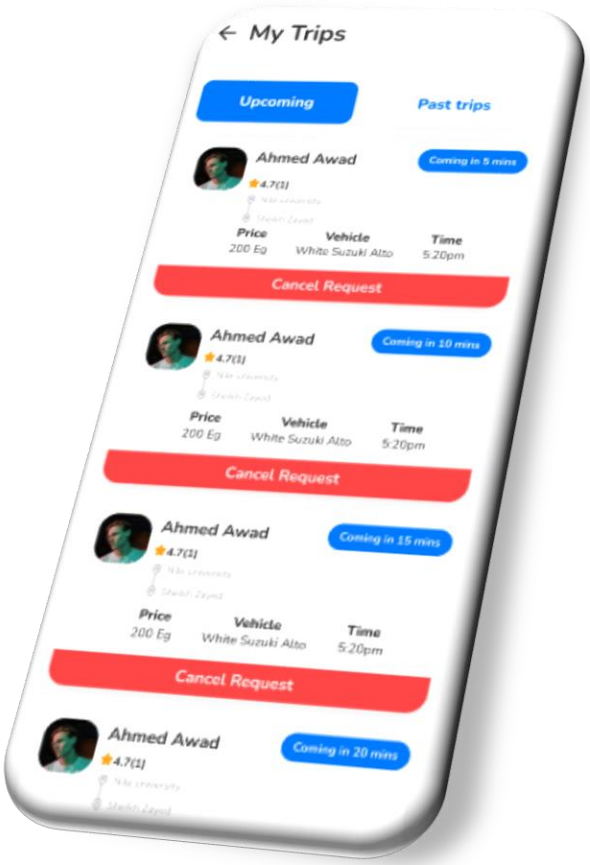


Figure 12 *Figure 13*

Figure 12: There will be a notification page to notify you with updates on your trip, or any application notification that you should be aware of.

Figure 13: showcases the Drawer Navigation, offering users quick access to key sections within the carpool application. The app features a user profile, My Trips, notifications, wallet information, and a secure logout option. It provides a centralized hub for managing trips, notifications, wallet status, and account actions, ensuring transparency and accountability.



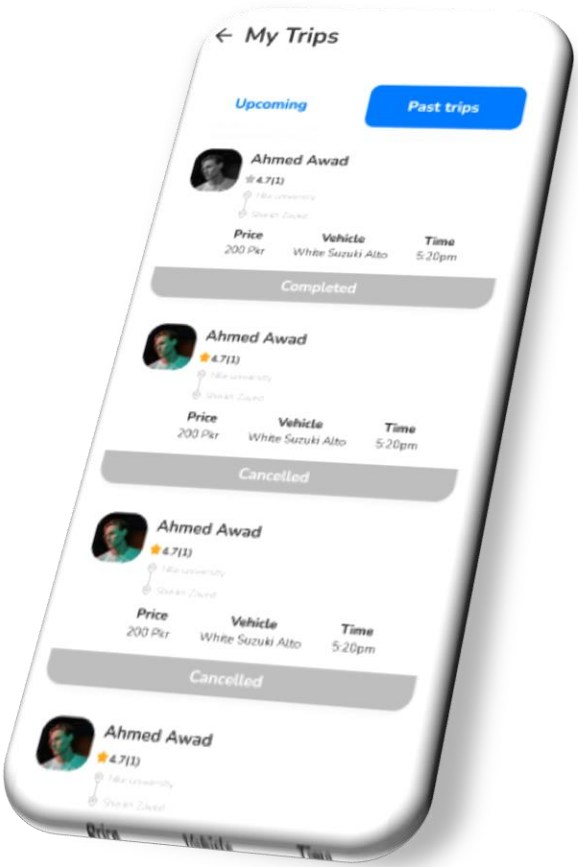


Figure 14 *Figure 15* *Figure 16*

Figure 14: providing users with a preview of their scheduled carpool journeys. The system displays a list of upcoming carpool trips, with status indicators and interactive options for users to view, modify, or cancel the trips, keeping them informed and manageable.

Figure 15: offering users a comprehensive view of their carpool history. The app provides a comprehensive history of carpooling journeys, allowing users to review their experiences, provide feedback, and access digital receipts for each completed trip.

5.2 Software interface:

The "GoPooling" mobile application is a cross-platform solution developed using the Flutter SDK for the frontend, Dart SDK for programming logic, and Android SDK tools for Android platform integration. The

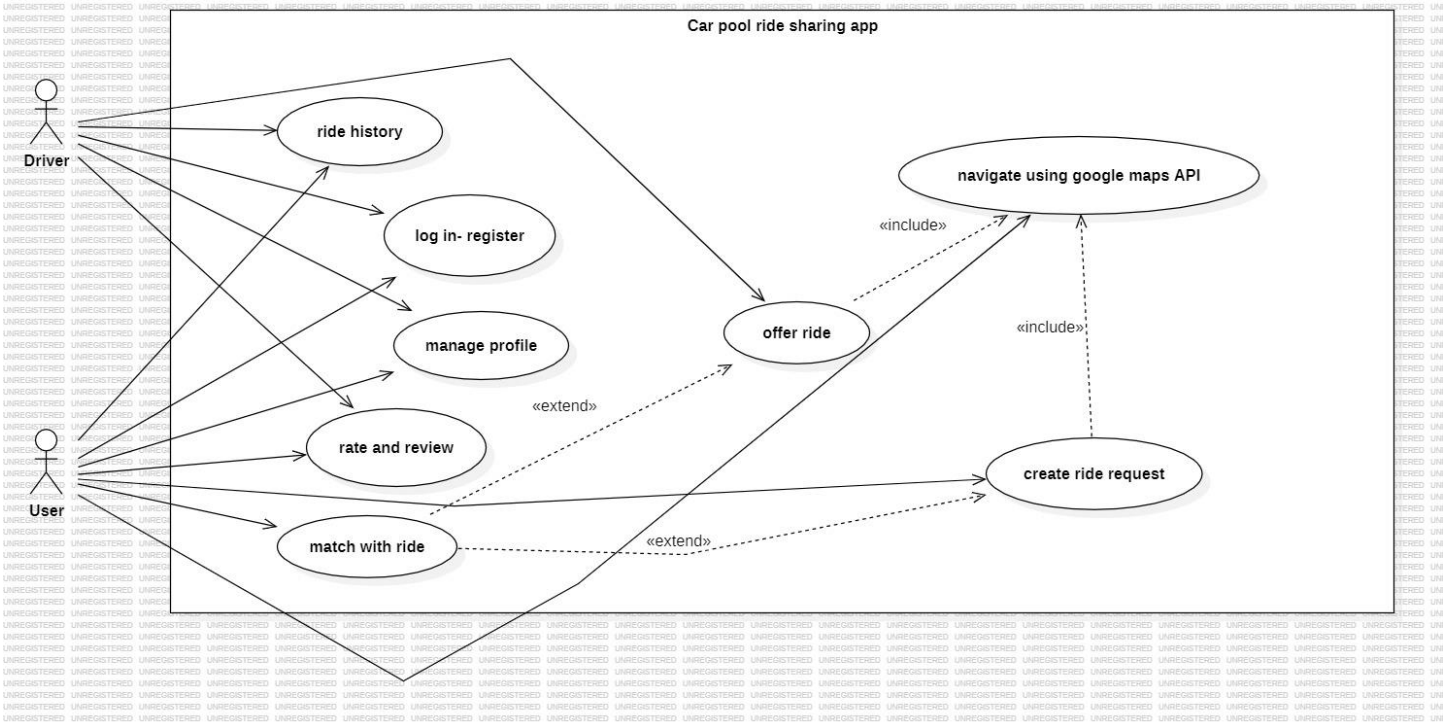
development process is facilitated through Visual Studio Code, a versatile and user-friendly integrated development environment.

5.3 Hardware interface:

"GoPooling" application will require a stable Internet connection to operate, Although The mobile phone will need to support GPS and internet connection.

6. Diagrams

6.1 Use case Diagram.



6.2 Use case scenarios

Use Case 1

Use Case Name	Book a Ride
Actors	User, Driver
Main Success Scenario	1. User logs in. 2. User enters ride details. 3. System matches ride request with available drivers. 4. User selects a ride offer. 5. System books the ride.
Exceptions	1. No drivers available. 2. User enters incomplete ride details. 3. System error occurs during booking.
Actions	1. Verify user login. 2. Validate ride details. 3. Match ride requests. 4. Confirm ride booking.
Pre-Condition	User must be registered and logged in.
Post Condition	Ride is booked, and user receives confirmation.

Use Case 2

Use Case Name	Offer a Ride
Actors	Driver
Main Success Scenario	1. Driver logs in. 2. Driver selects 'Offer Ride'. 3. Driver enters ride details. 4. System validates and posts the ride offer. 5. System notifies driver of user interest.
Exceptions	1. Incomplete ride details. 2. System error during posting. 3. No users show interest.
Actions	1. Verify driver login. 2. Collect ride details. 3. Validate and post offer. 4. Notify of user interest or lack thereof.
Pre-Condition	Driver must be registered, logged in, and have a valid driver's profile
Post Condition	Ride offer is posted, awaiting user response.

Use Case 3

Use Case Name	Rate and Review a Driver/Passenger
Actors	User, Driver
Main Success Scenario	1. User/Driver completes a ride. 2. User/Driver selects 'Rate and Review'. 3.User/Driver submits a review and rating. 4. System validates and posts the review.
Exceptions	1. Attempt to review before ride completion. 2. System error during review submission. 3. Review contains inappropriate content.
Actions	1. Confirm ride completion. 2. Collect review and rating. 3. Validate and post review.
Pre-Condition	User/Driver must have completed a ride.
Post Condition	Review is posted on the driver's/passenger's profile.

Use Case 4

Use Case Name	Manage Profile
Actors	User, Driver
Main Success Scenario	1. User/Driver logs in. 2. User/Driver selects 'Manage Profile'. 3. User/Driver updates profile information. 4. System validates and saves changes.

Exceptions	1. Invalid profile information. 2. System error during information update. 3. Unauthorized profile access attempt.
Actions	1. Verify user/driver login. 2. Display current profile information. 3.Validate new information. 4. Save changes to the profile.
Pre-Condition	User/Driver must be registered and logged in.
Post Condition	Profile information is updated and saved.

Use Case 5

Use Case Name	Cancel Ride
Actors	User, Driver
Main Success Scenario	1.User/Driver logs in. 2.User/Driver selects the booked ride. 3.User/Driver chooses to cancel the ride. 4.System confirms cancellation and notifies the other party.
Exceptions	1. Ride is too close to the start time for cancellation. 2. System error during cancellation process. 3. Ride does not exist or has already occurred.
Actions	1. Authenticate user/driver. 2.Retrieve upcoming rides from the database. 3. Display upcoming rides.

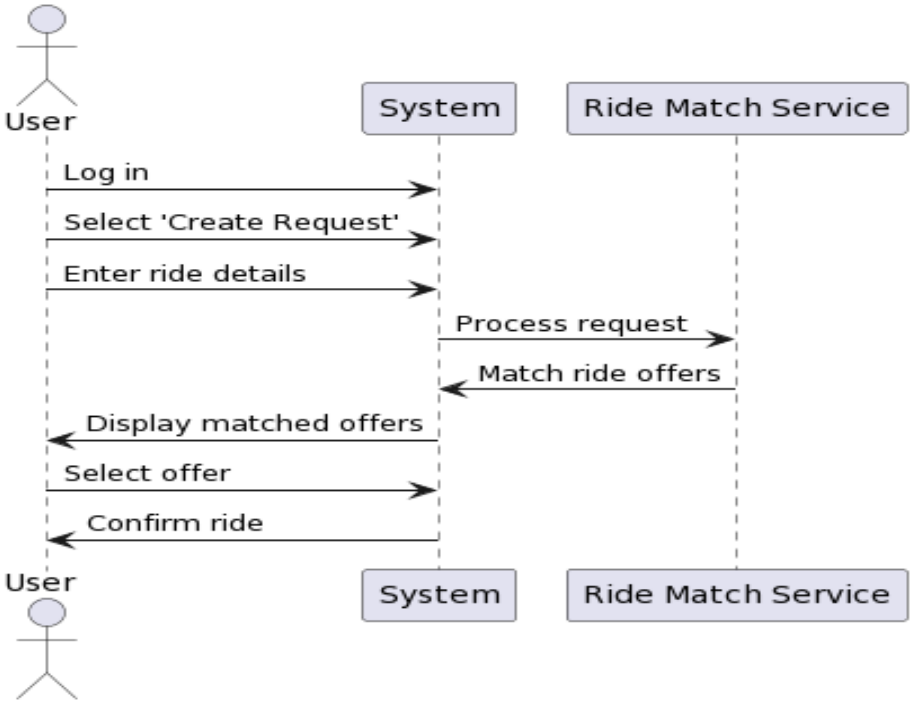
Pre-Condition	User/Driver must be registered and logged in.
Post Condition	User/Driver is informed about their upcoming rides.

Use Case 6

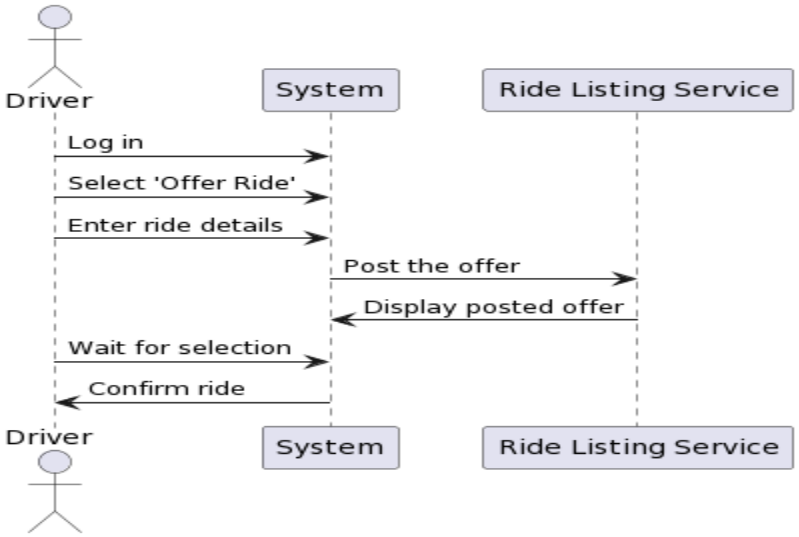
Use Case Name	Send Ride Notification
Actors	User, Driver
Main Success Scenario	1. System identifies a ride match. 2.System sends a notification to the User/Driver. 3. User/Driver acknowledges the notification.
Exceptions	1.User/Driver has disabled notifications. 2.Notification service is down.
Actions	1. Match rides based on system criteria. 2.Send out a notification through the app. 3. Log user/driver acknowledgement.
Pre-Condition	A ride match has been found based on user preferences.
Post Condition	User/Driver is aware of the ride match or offer.

6.3 Sequence Diagram

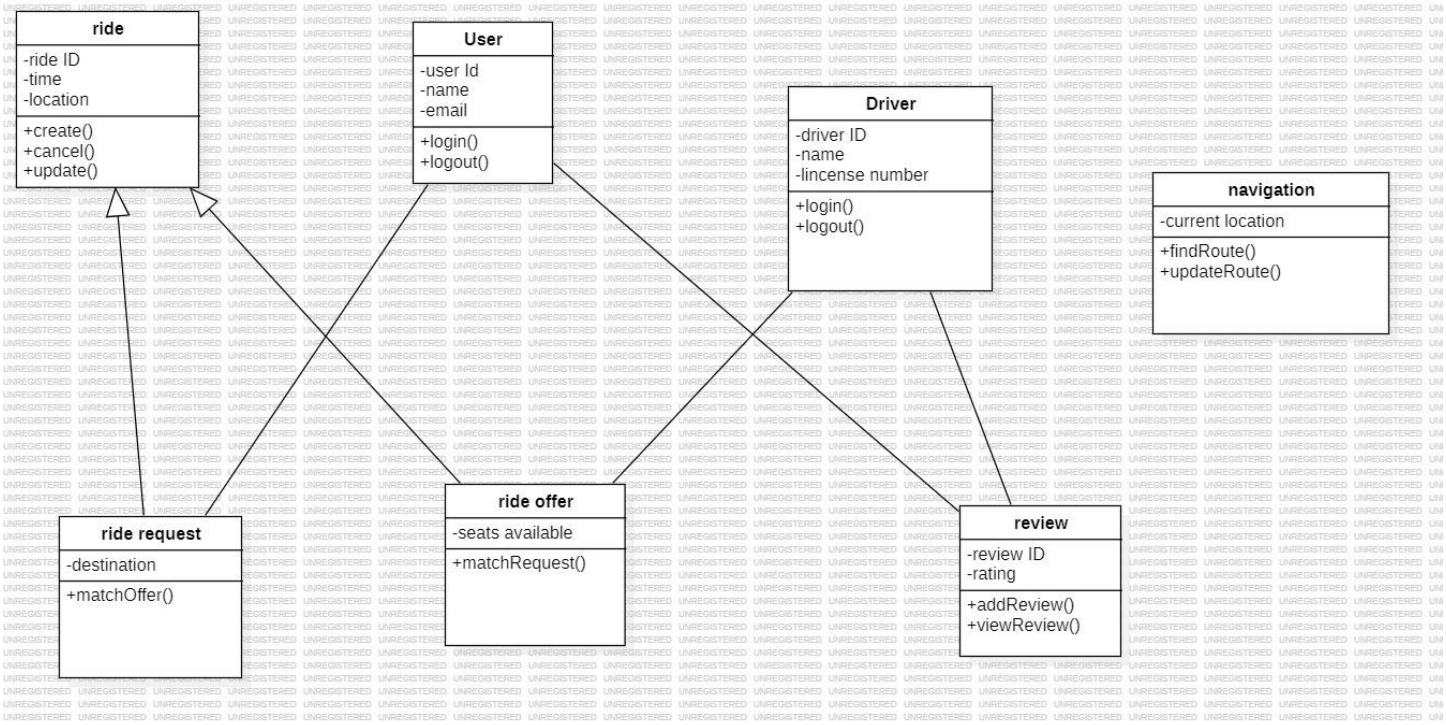
6.3.1 Ride match service



6.3.2 Ride listing service

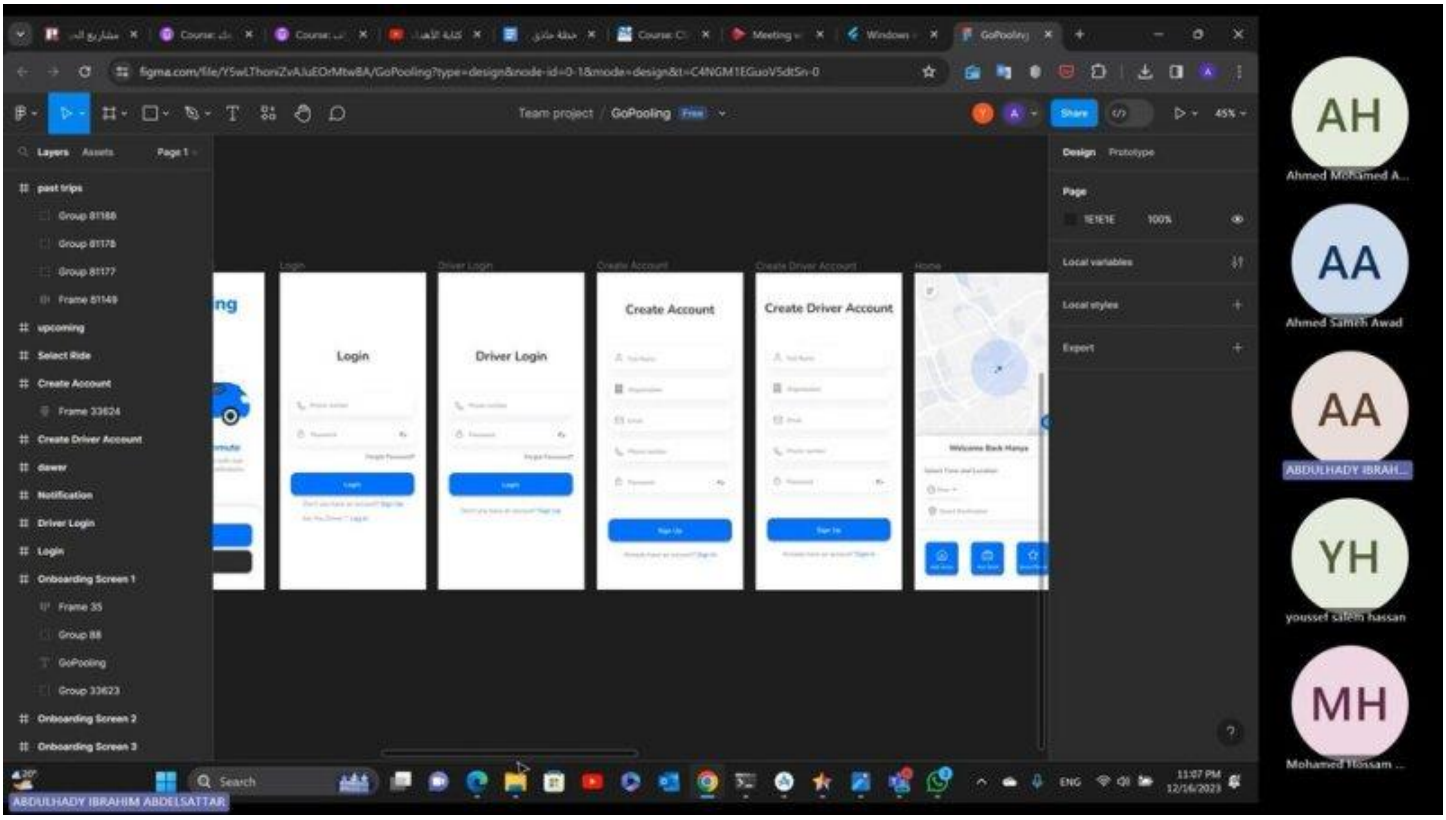


6.4 Class Diagram



6.5 Scrum retrospective video recording

[New channel meeting-20231216_230628-Meeting Recording.mp4 (sharepoint.com)]



7. Test Case Template

Project Name: GoPooling car ride sharing app.

Test Case Template

Test Case ID: T-01

Test Priority (Low/Medium/High): High

Module Name: User Authentication

Test Title: Valid Login Test

Description: To verify that the user can log in with valid credentials.

Test Designed by: Abdulhady & Youssef & Ahmed

Test Designed date: 12/25/2023

Test Executed by: Abdulhady & Youssef & Ahmed

Test Execution date: 12/25/2023

Pre-conditions: User has valid username and password, There is available rides.

Dependencies: User account must be pre-registered in the system

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
1	Navigate to login/Sign-up page	User= example@gmail.com	User should be navigated to the dashboard with successful login or To the sign-up page.	As Expected, User is navigated to the dashboard, or the sign-up page.	Pass
2	Provide a valid password	Password: 1234	User should be able to login using his Password.	As Expected, The user managed to log in the dashboard successfully using his provided Mail.	Pass
3	Provide a valid Mail	User= example@gmail.com Password: 1234	User should be able to login using his Mail.	As Expected, The user managed to log in the dashboard successfully using his provided password.	Pass
4	The User Shouldn't be able to use a logged in a mail for sign-up	User= example@gmail.com Password: 1234	Send an error message saying "Mail already logged in".	Not Expected, The user signed-up using his already logged in mail.	Fail
5	Booking a ride	User= example@gmail.com Password: 1234	The User should be able to book a ride.	The user successfully booked a ride.	Pass

Post-conditions:

1. After successful registration, the user's account information is stored securely in the database.
2. Upon successful login, the user is redirected to the home screen/dashboard.
3. After a ride is booked, the user receives a confirmation notification with details of the assigned driver and estimated arrival time.